Høyskolen Kristiania

Recommended steps before lecture

Read this article, click the link, and follow instructions:

- https://blogs.vmware.com/workstation/2024/05/vmware-workstation-pro-now-available-free-for-personal-use.html
- Create an account with Broadcom
- Make sure you select “free for personal use”
- Windows users download WORKSTATION, Mac users download FUSION

Follow this link to download VM image (for Windows users):

- https://www.osboxes.org/debian/#debian-10-vmware

For Mac users with M-type (ARM) CPU:

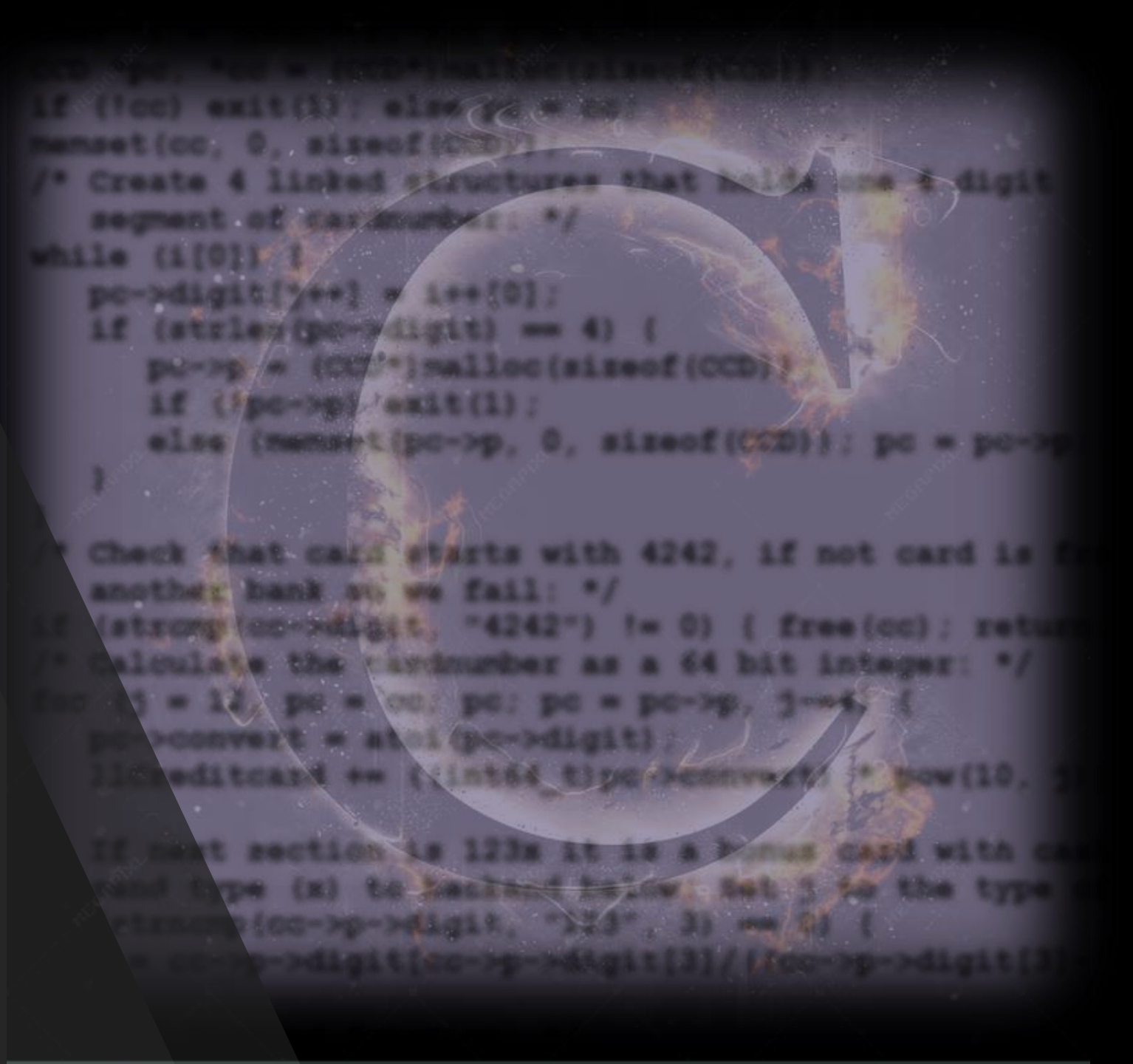
- https://www.eastwill.no/pg3401/Debian_PG34_ARM64.zip

Make sure you download the 64 bit version!

PG3401
C Programmering for Linux

Lecture 1 (week 02)

Introduction



In this lecture

- Structure of the course
- Introduction to C programming
- Setting up environment

About Bengt Østby

- C-programmer, specialist Windows system drivers
- Certified Ethical Hacker & IIA Internal Auditor
- 16 years experience from anti-virus industry
- Worked with advanced video surveillance
- Broken into banks and in critical infrastructure

- Lead Programmer, Norman
- Enterprise Architect Security CoE, AVG
- Security Concepts Group
- Head of Offensive Security, Capgemini Cybersecurity
- Nordic lead Advanced Attack and Readiness Operations, ACN
- Lecturer Høyskolen Kristiania (and USN)
- Senior Technologist - Stortinget

NORMAN® DATA DEFENSE SYSTEMS

EastWill Security

AVG

Høyskolen Kristiania

SCG

Capgemini

USN Universitetet i Sørøst-Norge

accenture security

Course structure

- Slides are in English, lectures will be held in English this year as there are international program students from Data Science

- 12 weeks of lectures and exercises
  - Lectures and lab exercises on campus
  - Check TimeEdit for dates and times
  - Course is built up of 12 lectures

- Exam
  - 2-week home exam
  - Exam will be practical with several problem-solving tasks, both chosen approach to a problem, code quality – and that it actually works will count towards your final grade

- Actual CODING is essential, there is only one way to learn to code… If you only attend lectures and no exercises you will not learn to code – only some theory :-)

Lecture Plan

- **Lecture 1** - Introduction to the course and Linux - Why C?
  Goal : -- Ability to use own Linux installation –
- Lecture 2 - Intro to Tools - OS tools, Compiling tools, Debugging tools
- Goal : -- Ability to write and compile a small C program–
- Lecture 3 - Short intro to C, Primitive Data types, Control structures
- Goal : -- Basic task solving ability –
- Lecture 4 - Pointers and applications
- Goal : -- Using memory better –
- Lecture 5 - Strings, Arrays and Structs
- Goal : -- Even more memories

Lecture Plan

- Lecture 6 - I/O - terminal and files
- Goal : -- A complete small single module program with functional I/O
- Lecture 7 - Module-based programming, function scopes, preprocessor
- Goal : -- Better organization of program into modules –
- Lecture 8 - Enumerated Types, Unions, Bit operations, Threads
- Goal : -- Just letting you know –
- Lecture 9 - Libraries, third-party sources, advanced debugging
- Goal : -- To get you not to reinvent the wheel –
- Lecture 10 – Networking in C
- Goal : -- Widening your scope –
- Lecture 11 – Safe Programming
- Goal : -- Practical and safe usage –
- Lecture 12 - Repetition and Summary

Timetable of lectures and exercises

Monday 5. January	12.15 – 16.00	Physical in Oslo (D-U101)	Lecture 1 : Introduction to the course, and Linux	
Monday 12. January	12.15 – 16.00	Physical in Oslo (D-U101)	Lecture 2 : Introduction to C and how to compile	
Monday 19. January	12.15 – 16.00	Physical in Oslo (D-U101)	Lecture 3 : Datatypes and control structures	
Monday 26. January	12.15 – 16.00	Physical in Oslo (D-U101)	Lecture 4: Pointers and applications	
Monday 2. February	12.15 – 16.00	Physical in Oslo (D-U101)	Lecture 5: Strings, arrays and structs	
Monday 9. February	12.15 – 16.00	Physical in Oslo (D-U101)	Lecture 6: I/O terminal and files	
Monday 23. February	12.15 – 16.00	Physical in Oslo (D-U101)	Lecture 7: Functions and preprocessor	*) Case study 21_3
Monday 2. March	12.15 – 16.00	Physical in Oslo (D-U101)	Lecture 8: Advanced topics; unions, bits etc	*) Case study 21_5
Monday 9. March	12.15 – 16.00	Physical in Oslo (D-U101)	Lecture 9: Libraries, 3rd party and threads	
Monday 16. March	12.15 – 16.00	Physical in Oslo (D-U101)	Lecture 10: Networking in C	*) Case study 21_6
Monday 23. March	12.15 – 16.00	Physical in Oslo (D-U101)	Lecture 11: Safe Programming	
Monday 13. April	12.15 – 16.00	Physical in Oslo (D-U101)	Exam preparation and course summary	

*) To get a clear understanding of what is needed for the exam we will go through an earlier exam, and show how to solve three of the tasks – including correct coding style (best practice), design, error handling, and coding comments

Streaming of lectures

- This course will NOT be streamed
- Lectures are held physical on campus in Oslo

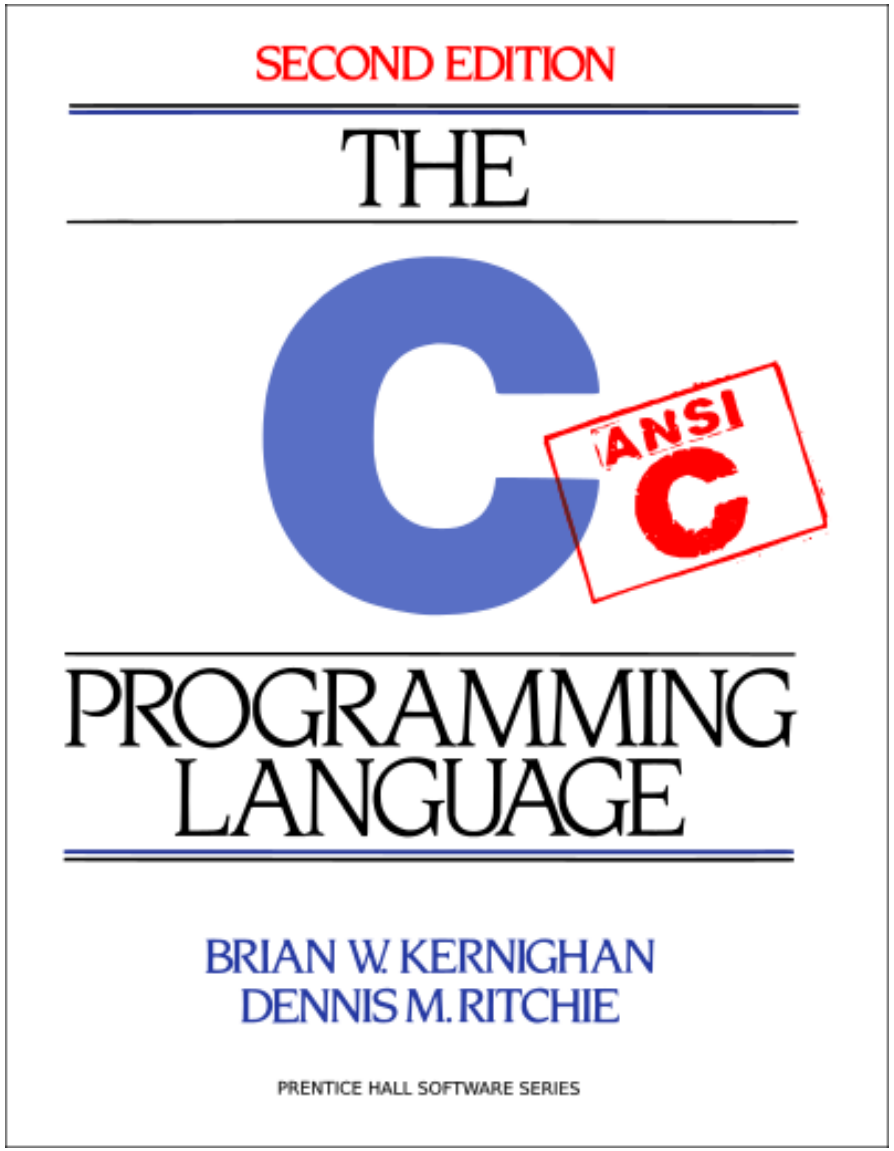
- Slides will be available on Canvas, also during the exam

Høyskolen Kristiania

Curriculum

Books?

- There are written thousand's of books on C
- K&R Second Edition is good
- Important to support «all» embedded compilers is to learn traditional C (before C99), that is the reason for this course teaching «ANSI C»
- Dennis Ritchie INVENTED the C programming language, so why read someone else's book on his language :-)



Høyskolen Kristiania

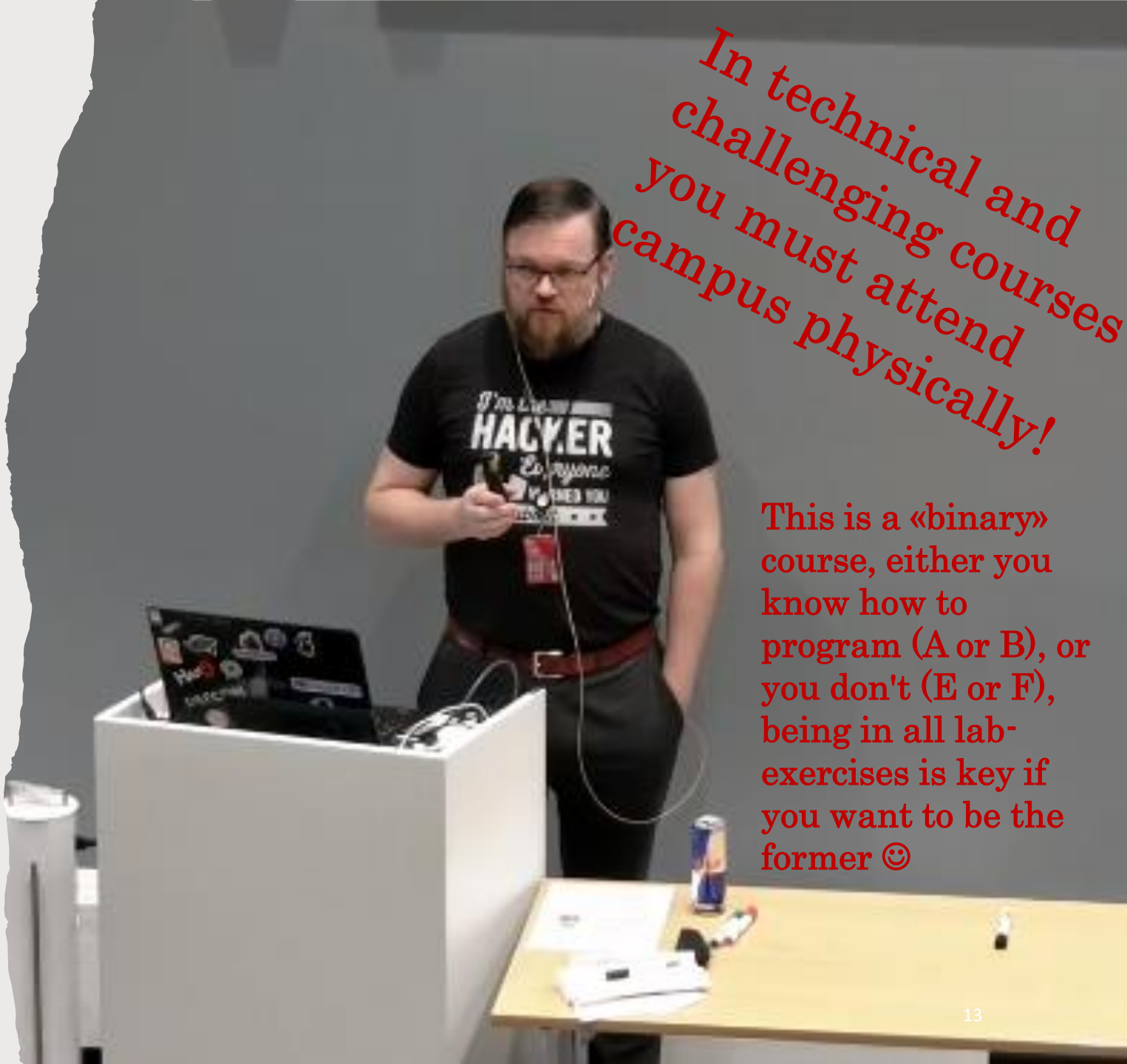
Lectures

- Lectures
- Exercises

You actually have to learn how to program to pass this course…

Working in Linux using an editor and compiler HANDS-ON is the only way to learn how to program!

Getting feedback from lecturer is the best way to do it well ☺



Options for environment

Course covers Linux, but the focus is on C programming.

Required version of Linux, recommend a preinstalled image:

Debian 10, 64 bit

This course IS named “C Programming for Linux” ☺

Previous years all versions of Linux has been supported, however this year we have 450 students taking C programming – so I need to streamline…

Also the exam this year might have tasks where you get a precompiled executable as part of the assignments ;-)

Recommend to run Linux in a virtual machine. To run a virtual machine, you have options:

(Debian 10 is not recommended to run on host – use a VM…)

Virtual Box (free for students)

Vm Ware Workstation (free for students)

Høyskolen Kristiania

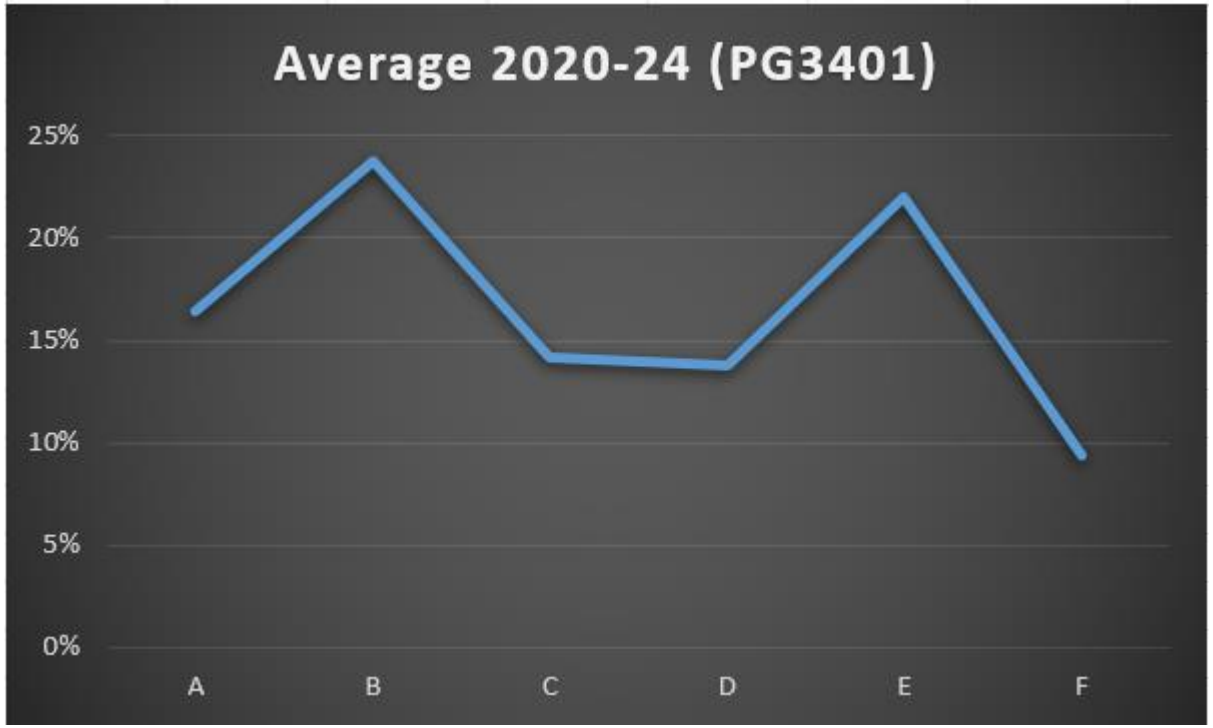
The exam

Exam format

- 2 week home exam, where 95% av exam is practical programming
- Exam can be answered in both English and Norwegian
- All tasks are stand-alone, which means that if you failed to complete a task you can still do the following tasks (in contrast to exams that have one big programming task)
- Expected task layout (subject to change):
  - Task 1 consists of 3 (simple) theory questions, and accounts for 5% of final grade.
  - Task 2 – 3 are fairly simple programming tasks.
  - Task 4 is typically creating a multithreaded application.
  - Task 5 is typically creating a network application.
  - Task 6 is typically a more difficult task, which might combine the above topics

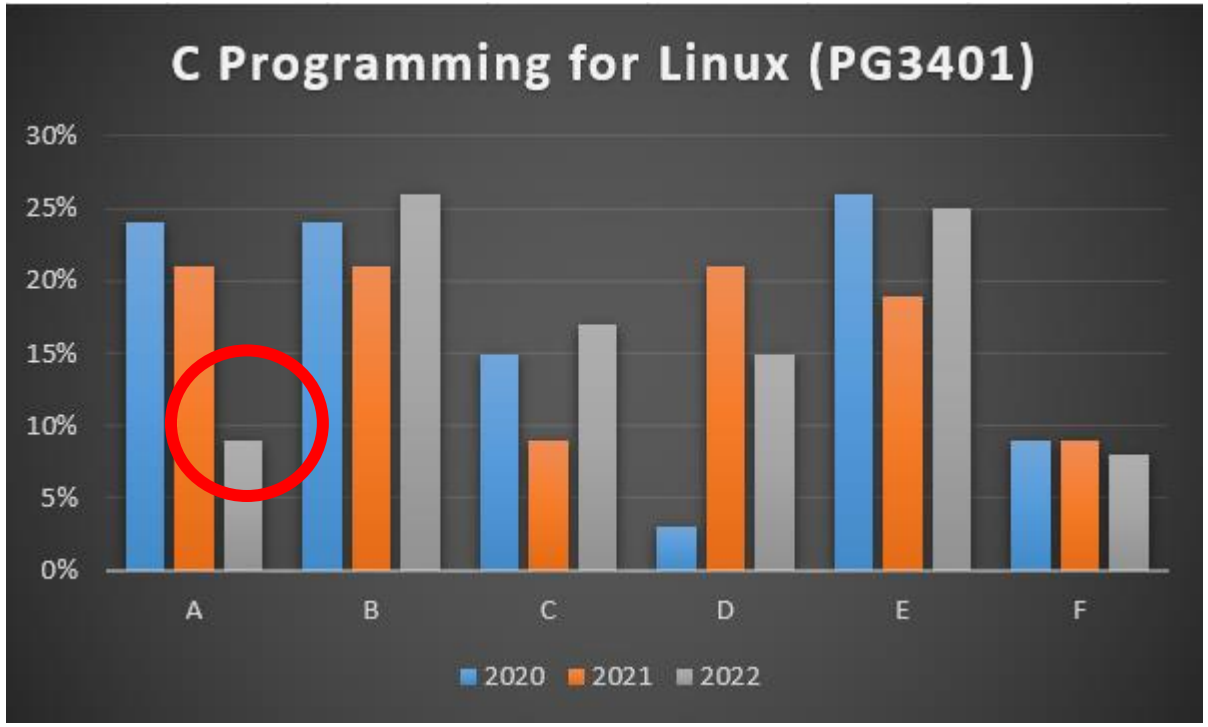
Exam format

- Note that C is a “binary” course, either you have learned how to code, or you have not
- From 5 years of experience teaching this class I can say with certainty that those who decide to not partake in the lab exercises end up with poor grades in this course
- Showing your code to the lecturer and getting feedback every week is the best way to master C programming!
- Notice that the curve is not the traditional “bell curve” common for exams; either you know how to code – or you don’t...



Importance of physical attendance

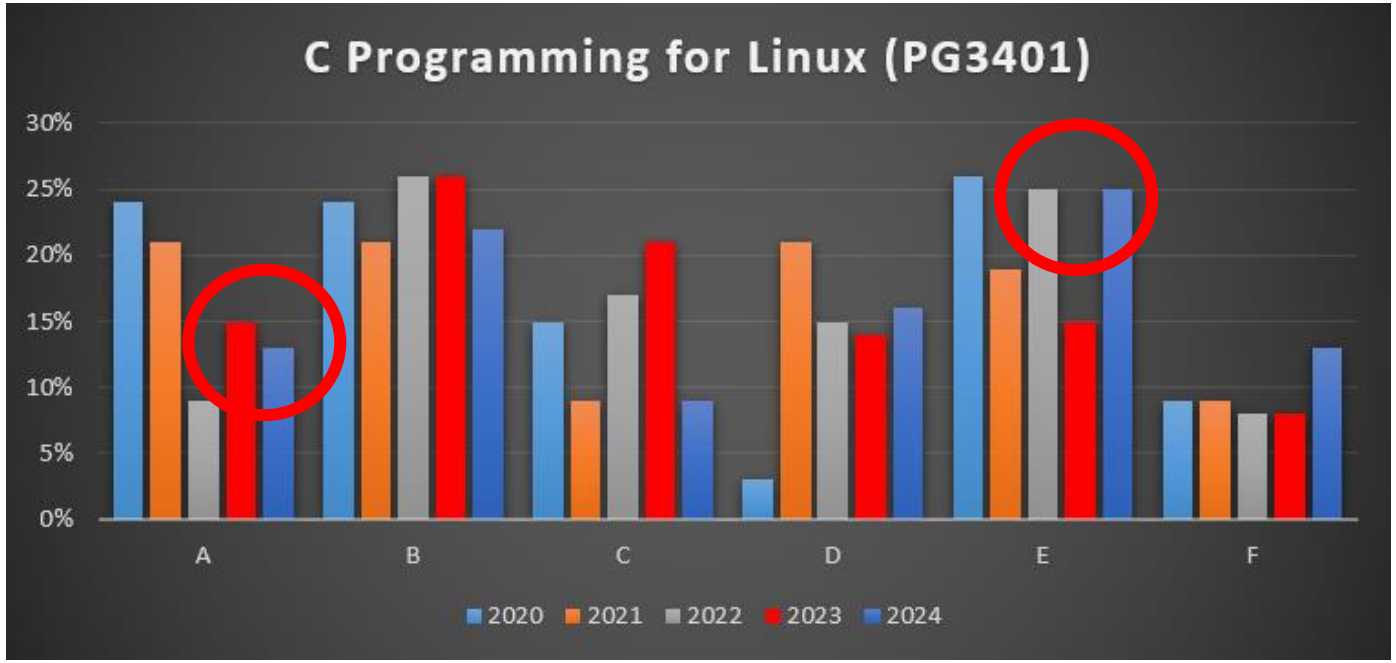
- Notice the dip in number of A's for the 2022-students
- Could be a statistical anomaly, but this was a class where the students have had year 1 and 2 of their bachelor during the pandemic, which seems to have been resulting in poor study habits
- We were also still streaming lectures that fall, reenforcing poor study habits and resulting in more students skipping class



Attending lectures and lab exercises helps you get good grades!

Importance of physical attendance

- Showing the last two slides has helped students understand the importance of physical attendance and working hard during lab exercises!



- The number of very good exams (A+B), has been steady and high
- However last year number of students attending lab exercises dropped again resulting in an increase in E grades...

The exam in this course is designed to not be feasible to answer using ChatGPT – planning to use AI will fail ;-)

Changes to exam format in 2025&2026

- At least 1-3 of the tasks on the exam will be on an “interactive” format, I have not created the exam yet – but planning to make a few tasks where solving the first parts of the task will give you instructions on how to complete the task
- This is a new way of combatting chatGPT-answers, it will not be more difficult than previous years (except if you are an AI...)
- An example of this will be demonstrated in a later lecture
- This means that this year you will get precompiled executables you need to run on your development system – so only the following solutions will be possible:

  - ***Linux Debian 10, 64 bit***
  - Running in a standard VM – either VMWare Workstation or Fusion

Høyskolen Kristiania

Linux

Høyskolen Kristiania

Operating System

- Linux, Windows, OSx, iOS, SintranIII, MS-DOS…

- Important roles of an operating system:
  - File Management
  - Process Management
  - Processing
  - Self organization
  - Handling hardware

See TK1104 on operating system if you need a recap.

In the beginning, there was UNIX

Not all true. In the beginning there was IBM mainframes (and they still exist…) Digital (VAX, PDP-11)

- Owned by AT&T, began in 1969
  - Copyright AT&T, but «poorly» enforced
  - Space Travel
  - "Mini"-computer
  - On the path to be de-facto standard in the 80s when they started becoming more restrictive with distributions
  - PC!
- Living descendants:
  - MacOSX
  - FreeBSD
  - OpenBSD
  - NetBSD
  - OpenSolaris

GNU is a free, cleanroom, implementation of Unix

- Richard Stallman
- 1983
- GPL
- Userland tools
- Never succeeded in making a kernel
- Worked on a modern microkernel

Linux is the kernel GNU never made!

- Linus Thorvalds,1991
- Contained a OS kernel and filesystem
- Userland tools from GNU and others
- Different “distributions"
- Business model:
  - Support
  - Warranties
  - Linux as part of devices **(embedded)**

- Hobby project based on a UNIX implementation

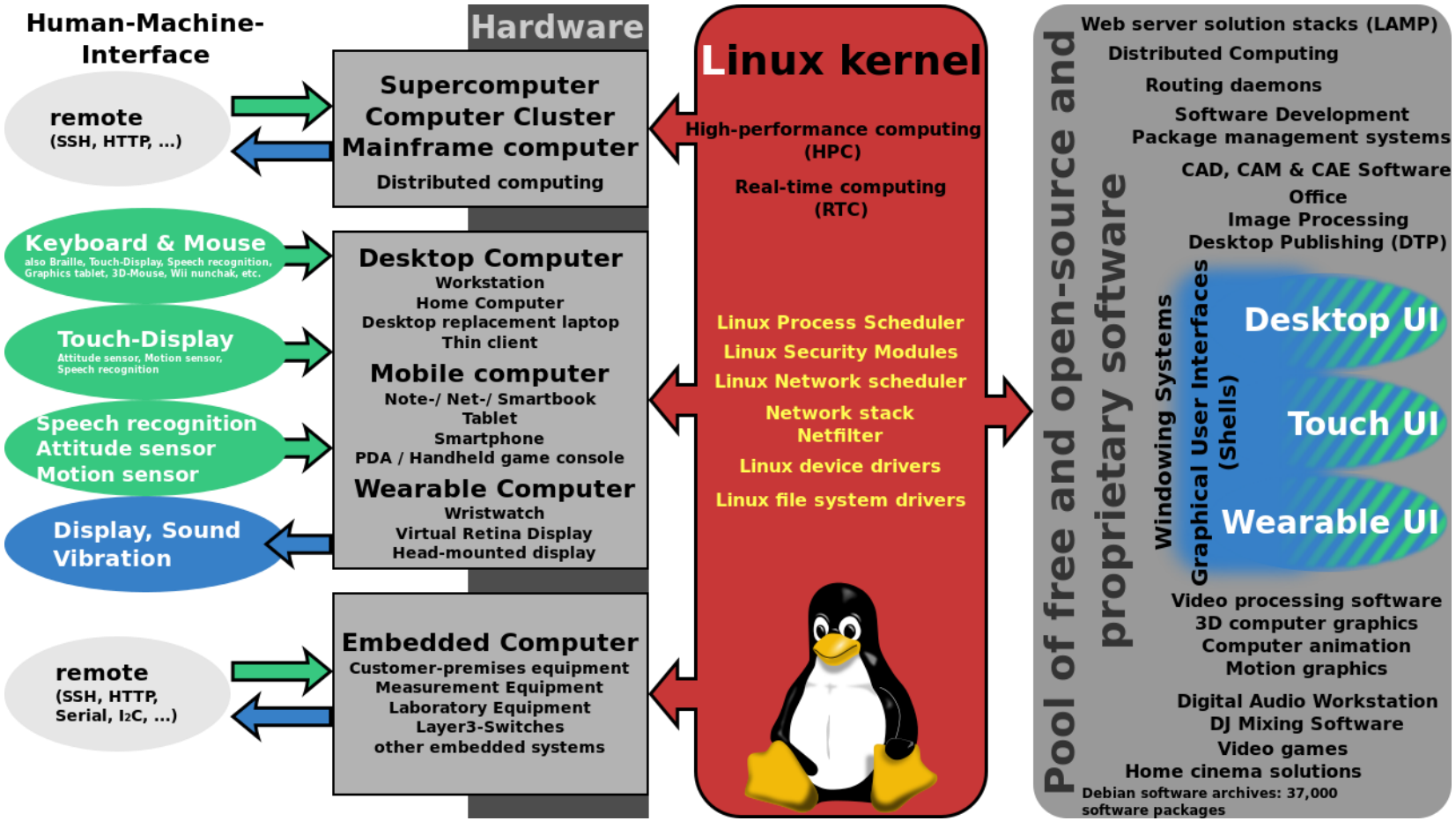
Linux is an extremely flexible OS

Google™

Very flexible OS

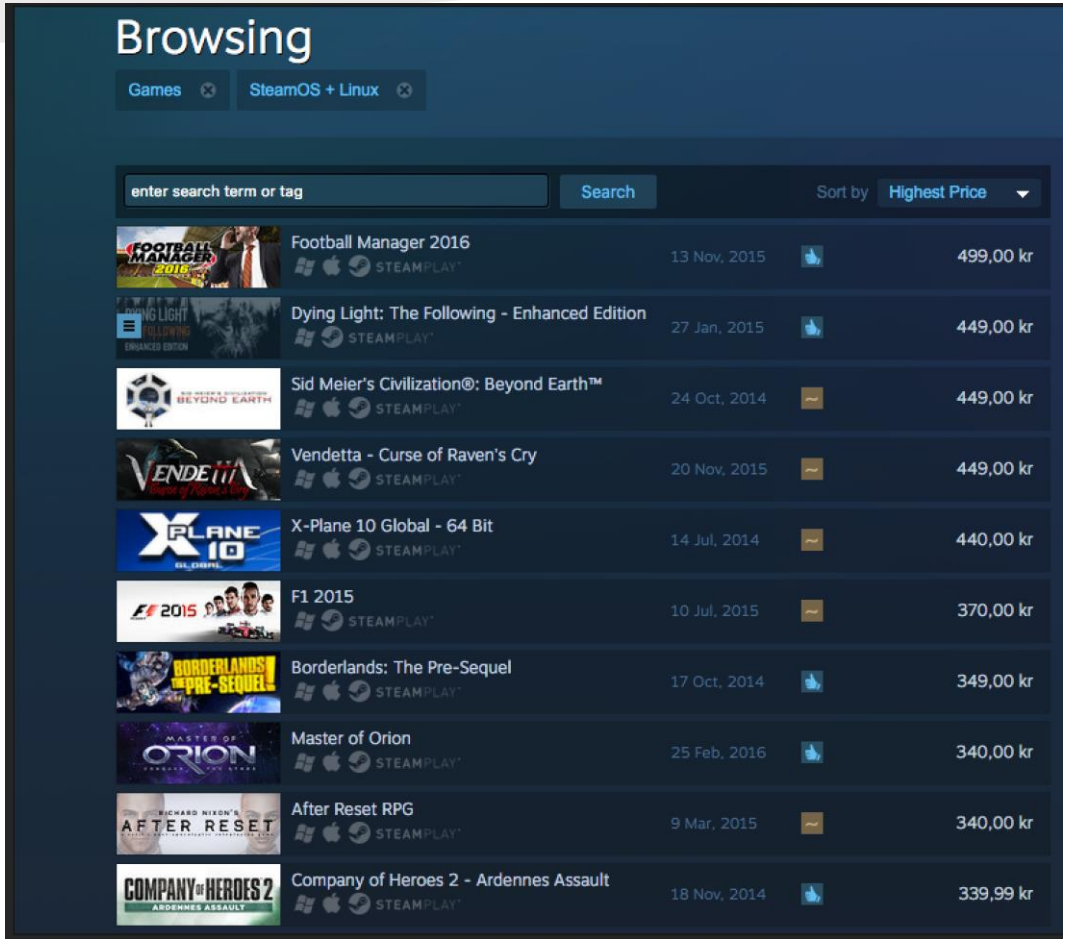
- Server
- Desktop
- Cluster
- Embedded

488 of top-500 super-computers run Linux


Human-Machine-Interface
remote
(SSH, HTTP, ...)
Keyboard & Mouse
also Braille, Touch-Display, Speech recognition, Graphics tablet, 3D-Mouse, Wii nunchak, etc.
Touch-Display
Attitude sensor, Motion sensor, Speech recognition
Speech recognition
Attitude sensor
Motion sensor
Display, Sound
Vibration
remote
(SSH, HTTP, Serial, I2C, ...)
Hardware
Supercomputer
Computer Cluster
Mainframe computer
Distributed computing
Desktop Computer
Workstation
Home Computer
Desktop replacement laptop
Thin client
Mobile computer
Note-/ Net-/ Smartbook
Tablet
Smartphone
PDA / Handheld game console
Wearable Computer
Wristwatch
Virtual Retina Display
Head-mounted display
Embedded Computer
Customer-premises equipment
Measurement Equipment
Laboratory Equipment
Layer3-Switches
other embedded systems
Linux kernel
High-performance computing (HPC)
Real-time computing (RTC)
Linux Process Scheduler
Linux Security Modules
Linux Network scheduler
Network stack
Netfilter
Linux device drivers
Linux file system drivers
Pool of free and open-source and proprietary software
Web server solution stacks (LAMP)
Distributed Computing
Routing daemons
Software Development
Package management systems
CAD, CAM & CAE Software
Office
Image Processing
Desktop Publishing (DTP)
Windowing Systems
Graphical User Interfaces (Shells)
Desktop UI
Touch UI
Wearable UI
Video processing software
3D computer graphics
Computer animation
Motion graphics
Digital Audio Workstation
DJ Mixing Software
Video games
Home cinema solutions
Debian software archives: 37,000 software packages

Høyskolen Kristiania

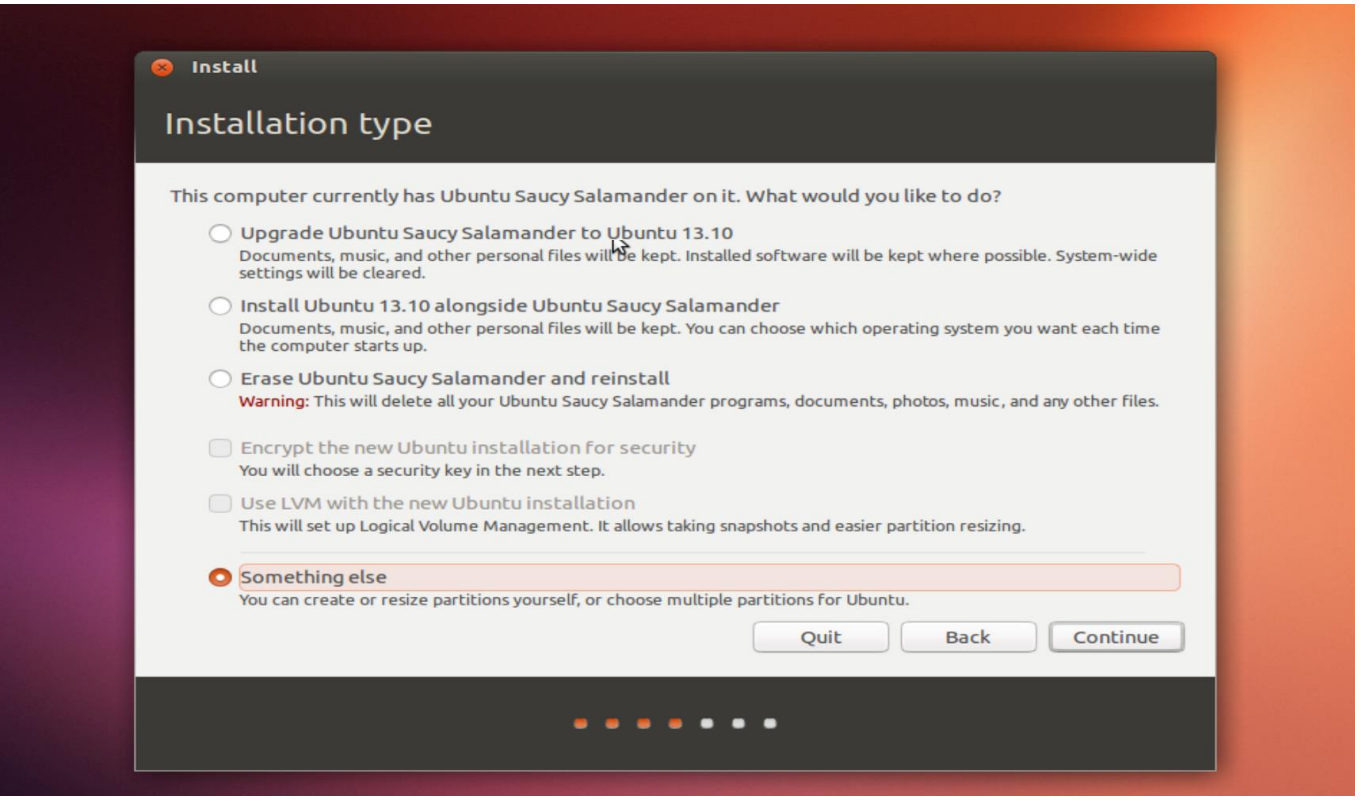
Now with games



The General Public License allows redistribution, but requires you to grant the same rights to all

Different lisensing models:
GPL v2, GPL v3
MIT, BSD, Apache

What is a “Linux Distribution”?



Flesh, bells and whistles

Installers

- Ubuntu now is the most popular
- Mint[Ubuntu with simpler desktop]
- Gentoo[The hippie],
- Debian[Old school],
- Slackware[Even older school]
- Fedora[Death Bed],
- CentOS/Red-Hat[Death Bed],
- Suse[Dead]

Cons: Distributions come and go and differ in terms of what kind of CLIB they support. When I started doing this, RedHAT was the big thing, then came Suse (which isn't that popular anymore) etc. In Norman we only had to support some distros, and then detect what we installed on...

To be good at Linux you need to be good with terminal emulators


```
hka@westerdals: ~/westerdals
hka@westerdals:~/westerdals$ ls -l | grep ".c"
-rw-r--r-- 1 hka hka  193 Jul  3 08:24 hello2.c
-rw-r--r-- 1 hka hka   74 Jul  3 08:22 hello.c
hka@westerdals:~/westerdals$ ls *.c
hello2.c  hello.c
hka@westerdals:~/westerdals$ ls -la *.c
-rw-r--r-- 1 hka hka 193 Jul  3 08:24 hello2.c
-rw-r--r-- 1 hka hka  74 Jul  3 08:22 hello.c
hka@westerdals:~/westerdals$ nano
hka@westerdals:~/westerdals$ ls
hello  hello2.c  hello.c  test.txt
hka@westerdals:~/westerdals$ cat test.txt
fasfasfsa
adsfasfdsaf

hka@westerdals:~/westerdals$ rm test.txt
hka@westerdals:~/westerdals$
```

“Shell” is way to interact with OS

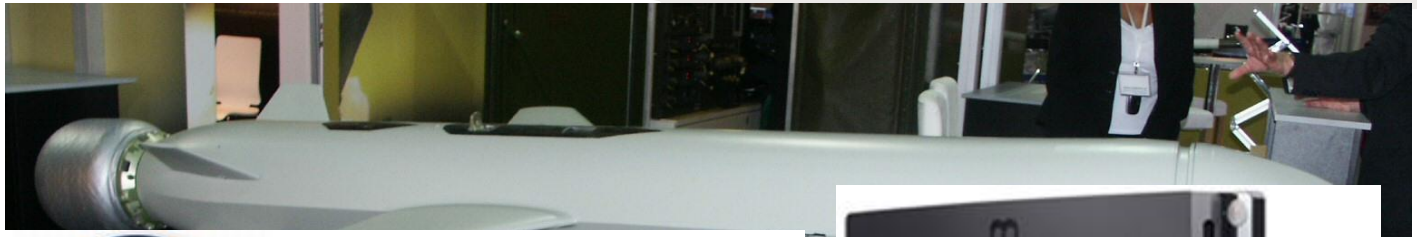
Most common:

- BASH [Bourne Again SHell]
- CSH [C SHell]
- GUI available but we live in denial!

Høyskolen Kristiania

C programming language

C is used EVERYWHERE, but you might not “C” it!



Linux itself (like Unix) is written mainly in C (some assembly in the kernel, but very little, maybe 2%)

The most popular programming languages?

https://www.tiobe.com/tiobe-index/

Aug 2020	Aug 2019	Change	Programming Language	Ratings	Change
1	2	↑	C	16.98%	+1.83%
2	1	↓	Java	14.43%	-1.60%
3	3		Python	9.69%	-0.33%
4	4		C++	6.84%	+0.78%
5	5		C#	4.[illegible]%	+0.[illegible]%
6	6		Visual Basic	4	
7	7		JavaScript	2	
8	20	⇈	R	2	
9	8	↓	PHP	2	
10	10		SQL	1	
11	17	⇈	Go	1	
12	18	⇈	Swift	1	
13	19	⇈	Perl	1	
14	15	↑	Assembly language	1	
15	11	⇊	Ruby	1	
16	12	⇊	MATLAB	0	
17	16	↓	Classic Visual Basic	0	

Dec 2022	Dec 2021	Change	Programming Language	Ratings	Change
1	1		Python	16.66%	+3.76%
2	2		C	16.56%	+4.77%
3	4	↑	C++	11.94%	+4.21%
4	3	↓	Java	11.82%	+1.70%
5	5		C#	4.92%	-1.48%
6	6		Visual Basic	3.94%	-1.46%
7	7		JavaScript	3.19%	+0.90%
8	9	↑	SQL	2.22%	+0.43%
9	8	↓	Assembly language	1.87%	-0.38%
10	12	↑	PHP	1.62%	+0.12%

Embedded programming is making C popular (again)…

What is C?

- Programming Language…
- Brian Kernighan and Dennis Ritchie (K&R “C”)
- Powerful programming language

- Easy to write *compilers*
- *The compiler of C is written in C!*
- *The operating system is written in C!*
- *Most high-level languages use interpreters, written in C*

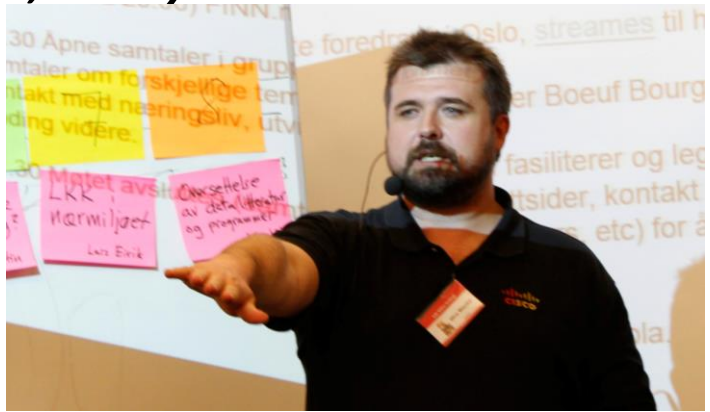
- With great power comes great responsibility

The philosophy of C

- Trust the programmer
- Keep the language small and tidy
- Provide only one way to do an operation
- Make it fast, even if it’s not guaranteed to be portable
- Maintain conceptual simplicity
- Don’t prevent the programmer from doing what needs to be done.

"Programming is hard. Programming correct C and C++ is particularly hard. Indeed, both in C and certainly in C++, it is uncommon to see a screenful containing only well defined and conforming code. Why do professional programmers write code like this? ***Because most programmers do not have a deep understanding of the language they are using.*** *While they sometimes know that certain things are undefined or unspecified, they often do not know why it is so."*

Olve Mauldal



Who does C?

- Performance is priority
- Real-time system developers
- Limited resources – embedded systems
- Operating systems and drivers
- Parts of many AAA-games

- On a full and rich OS you can develop much faster in a high level language – but you can only develop what that language support!
- In C you can make everything, it will (or can) be fast and without overhead, but it is hard…
- If you are close to hardware, then you code in C

Why learn?

- To be aware of pitfalls
- Ability to develop big projects with performance priority
- Reverse engineering
- Being forced [CUDA, OpenGL, Ffmpeg, pthreads]

Some of you will go on to learn C++, which is a fully modern language based on C.

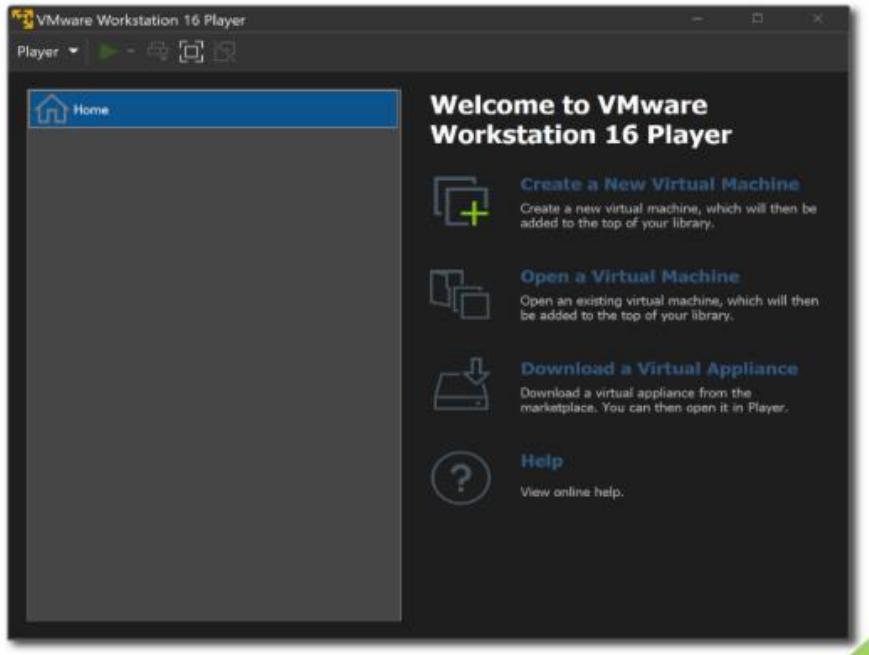
- C then C++
- C++ is based on C
- C++ does not make sense without C
- C is easier to learn
- C gives a great perspective on the Object Oriented paradigm

- Objective-C is used for mobile development, which is also a variant of C

- *(Don’t ever try to use C– for anything…)*

Høyskolen Kristiania

This course uses Virtual Machines



Virtualization allows running multiple simultaneous OS’es on the same HW

- Send finished installations around
- For testing and experiments (snapshots…)
- Popular with (anti-)virus programmers
- Consolidating servers
- The basis of Cloud computing (VM-ware, AWS, Azure…)

Virtualization comes with its own terminology

- Host operating system
- Hypervisor
- Virtual Machine
- Guest operating system
- Snapshot

Høyskolen Kristiania

This course will use VM-Ware

https://blogs.vmware.com/workstation/2024/05/vmware-workstation-pro-now-available-free-for-personal-use.html

- VMWare Workstation is an ideal utility for running a single virtual machine on a Windows or Linux PC. Organizations use VMWare Workstation to deliver managed corporate desktops, while students and educators use it for learning and training.
- The free version is available for non-commercial, personal and home use. **Commercial organizations require commercial licenses to use VMWare.**

Høyskolen Kristiania

This is the luddite course

```
Welcome to Ubuntu 16.04.1 LTS (GNU/Linux 4.4.0-31-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/advantage

7 packages can be updated.
0 updates are security updates.


*** System restart required ***
Last login: Mon Aug 15 11:36:10 2016 from 81.175.52.98
kjetilr@oslo:~$ ls
kjetilr@oslo:~$ ls /etc/
X11                     console-setup   fstab            inputrc          libaudit.conf   mdadm
acpi                    cracklib        fuse.conf        insserv          libnl-3         mime.types
adduser.conf            cron.d          gai.conf         insserv.conf     lintianrc       mke2fs.conf
alternatives            cron.daily      groff            insserv.conf.d   locale.alias    modprobe.d
apm                     cron.hourly     group            iproute2         locale.gen      modules
apparmor                cron.monthly    group-           iscsi            localtime       modules-load.d
apparmor.d              cron.weekly     grub.d           issue            logcheck        mtab
apport                  crontab         gshadow          issue.net        login.defs      nanorc
apt                     crypttab        gshadow-         kbd              logrotate.conf  network
at.deny                 dbus-1          gss              kernel           logrotate.d     networks
bash.bashrc             debconf.conf    hdparm.conf      kernel-img.conf  lsb-release     newt
bash_completion         debian_version  host.conf        krb5.conf        ltrace.conf     nsswitch.conf
bash_completion.d       default         hostname         krb5.conf.old    lvm             ntp.conf
bindresvport.blacklist  deluser.conf    hosts            krb5.keytab      machine-id      opt
binfmt.d                depmod.d        hosts.allow      ld.so.cache      magic           os-release
byobu                   dhcp            hosts.deny       ld.so.conf       magic.mime      overlayroot.conf
ca-certificates         dpkg            init             ld.so.conf.d     mailcap         pam.conf
ca-certificates.conf    environment     init.d           ldap             mailcap.order   pam.d
calendar                fonts           initramfs-tools  legal            manpath.config  passwd
kjetilr@oslo:~$
```

- Luddite? Those who sabotaged industrial weaves under the industrial revolution because machines took all the jobs
- Most Linux users don't use graphical tools, only Linux shell and different colors

- I am «less» nerdy than many others – I actually use Windows in addition to Kali Linux ☺

What can C be used for?

C is very flexible, everything can be created using C!

Other languages might be faster to develop, but nothing is as powerful and flexible as C...

– how to detect cheats on exams?

Problem:

- ***When grading Ethical Hacking exam I found a screenshot I did not understand...***
- ***Then I found the same screenshot a few days later***
- ***Inspecting in more detail I saw that instead of logging into my test machine running Apache the students was logging into a "fake" machine with Nginx!***

What can C be used for?

Problem:

- ***Documentation on the exam was screenshots in a PDF file, I wanted to see if there were others I had missed...***

Solution:

3 days of work, 700 (new) lines of C-code

- ***File enumerator that finds all PDF files in a directory tree***
- ***PDF format parser that finds all images in the PDF file***
- ***Decompressing ZLIB compressed data, and manually creating a PPM image header to store the images on disk***
- ***Using OCR to read text in the images to make them "searchable"***
- ***Searching for the word "nginx" in all exams (' images)***

We will learn to be “old-school”

- For those familiar with modern IDE, coding and compiling on command-prompt on Linux will feel like exchanging a modern drill with a box of screwdrivers
- But you will all thank me when you work with embedded systems (or kernel mode drivers)
- Debugging crashes on a machine without a keyboard or monitor, or a system that crashes during boot sequence - is extremely difficult, but possible as long as we stay «old-school»

Høyskolen Kristiania

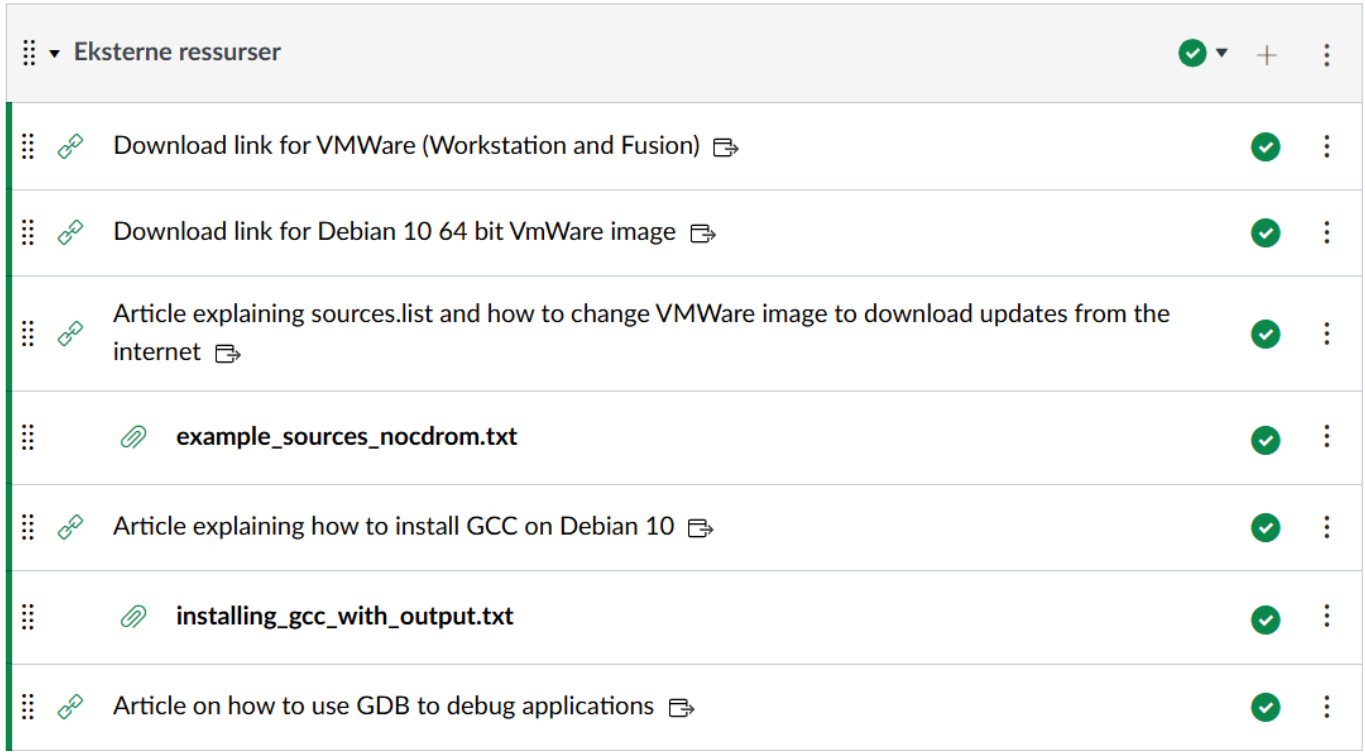
Today's assignment

- Install Linux
- Run Linux
- Make account with new password
- Delete old account
- Configure as you like it (keyboard layout??)
- Surf the web, check mail
- Try Gedit

Practical – Installing

- **Install VmWare Workstation or Fusion**
- **Install Debian 10 64 bit image**
- **Perform some simple tasks**

- **Links found in Canvas:**



Practical – Linux

- **wget http://www.eastwill.no/pg3401/linux.txt**

- Try to read the file
- Open a new tab in terminal
- Follow the instructions from the file

Lab exercises

- **Help each other becoming used to Linux. Some of you might have used it before, some are new to this OS. Make sure you all help each other – this is one of the most difficult courses you will take at this school, everyone of you will need help from the class! :-) You must attend Campus trainings, trying to learn it all your self will most often fail, hard…**

- **After next week everyone in the class MUST have a running Linux environment and be able to edit and compile code!**

- **NOTE: Do not copy/paste source code from slides, Powerpoint will have formatted a double-quote (") and other characters (for instance -), and it is almost guaranteed to insert extra spaces when you copy.**
- **Instead WRITE the source code you see on the slides into your editor...**

Lab exercises

Next week we will install GCC, a compiler for Linux, and we will learn more about how to use Linux – so what we have done is enough for today ☺

I require the use of Debian 10 and GCC, earlier classes has had students trying to use other tools and other Linux distros, and some ended up with very strange problems I had problems solving…

With 200+ students ONLY Debian 10 will be supported this year (will also be required during the exam)

Lab exercises

See more detailed explanation to today's lab exercise in

PG3401_Exercises_01_MacFusionWmdk.pdf

PG3401_Exercises_01_WindowsVmware.pdf

Høyskolen Kristiania